

External Ternary Judge Adapter for OpenClaw and DeepSeek V4 API

Engineering Whitepaper

Version 1.0 - Technology Assessment Draft

Prepared: July 9, 2026

Core engineering thesis

A standard LLM should propose candidate reasoning moves, but a separate external module should decide which moves may enter the accepted state. The judge classifies each move as FOLLOW, OMIT, NULL, or UNDECIDED under an explicit verification contract.

Confidential working draft - not a product specification

Document Control

Field	Value
Document type	Engineering whitepaper
Scope	External judge adapter for OpenClaw runtime and DeepSeek V4 API
Primary goal	Evaluate whether a verification middleware can reduce false accepted reasoning steps
Primary metric	FalseFollowRate
Primary deployment mode	OpenClaw sidecar skill / middleware with explicit state manager
Model dependency	DeepSeek V4 API as generator and structured extractor; judge authority remains external

1. Executive Summary

This whitepaper specifies a practical architecture for implementing an external ternary judge adapter on top of OpenClaw and the DeepSeek V4 API. The system is intended as a technology assessment platform, not as a production autonomous agent. Its purpose is to evaluate whether a separate verification middleware can reduce state-corrupting reasoning moves produced by a conventional large language model.

The core system rule is: the LLM proposes, the adapter normalizes, the judge certifies, and the state manager commits only certified updates. A generated statement, retrieved passage, tool call, or reasoning step is treated as a candidate move. Each candidate is classified as FOLLOW, OMIT, NULL, or UNDECIDED under an explicit verification contract.

Verdict	Meaning	Controller action
FOLLOW	Certified productive state refinement.	Commit the transition to accepted state.
OMIT	Admissible but semantically invariant or inessential.	Drop, compress, or log as redundant.
NULL	Certified inadmissibility, contradiction, invalid operation, or nullity.	Block the move and prevent state entry.
UNDECIDED	Certification is insufficient under the current contract.	Retrieve, verify, route, defer, or abstain.

OpenClaw is suitable as the runtime shell because its documentation describes a self-hosted gateway connecting chat apps to AI coding agents, with tool use, sessions, memory, and multi-agent routing [R1]. DeepSeek V4 is suitable as the hosted model backend because its API exposes

OpenAI- and Anthropic-compatible formats and model names such as deepseek-v4-flash and deepseek-v4-pro [R5].

2. Problem Statement

A standard LLM does not maintain a clean distinction between plausible text and certified state transition. In ordinary agent loops, text produced by the model often becomes part of the mutable transcript and later influences tool calls, memory writes, retrieval decisions, and final answers. This is structurally unsafe: a fluent unsupported statement can become accepted context.

The project addresses this failure by introducing a pre-commit verification layer. No claim, retrieved passage, tool action, or reasoning step is allowed to enter the accepted state solely because the generator produced it.

Generator output $\neq$ accepted state

Accepted state = committed certified transitions only

2.1 Non-goals

- The first prototype is not a universal natural-language logic engine.
- The first prototype does not require training or modifying DeepSeek model weights.
- The judge does not replace RAG, theorem provers, solvers, or tool verification.
- The judge is not a preference model and should not be trained primarily on stylistic human preference data.
- The project should not initially target arbitrary open-ended chat; it should start with controlled RAG and computable contracts.

3. Source Platform Assumptions

3.1 OpenClaw Runtime Assumptions

The architecture assumes OpenClaw is used as a self-hosted agent gateway rather than as the judge itself. OpenClaw documentation describes it as a gateway across many chat apps and channel plugins; the Gateway is described as the single source of truth for sessions, routing, and channel connections [R1]. The documentation also lists agent-native capabilities including tool use, sessions, memory, and multi-agent routing [R1].

OpenClaw skills are markdown instruction files contained in directories with SKILL.md. The skills system loads workspace skills, project agent skills, personal skills, managed local skills, bundled skills, and extra directories with priority rules [R2]. This makes a judge adapter naturally deployable as one or more OpenClaw skills.

OpenClaw also supports isolated agents and per-agent sessions. Its multi-agent documentation states that configured agent IDs can represent fully isolated personas with separate channel accounts, personalities, auth, and sessions unless explicitly shared [R3]. This is useful because different verification contracts should run in isolated agents or workspaces.

OpenClaw capability	Use in this project
Gateway and channel plugins	Expose the judge-enabled agent through chat channels or WebChat.
Sessions and routing	Bind each task to a persistent verification state.
Skills	Package adapter, judge, RAG verifier, and state manager as skills.
Multi-agent routing	Route math, RAG, code, and planning tasks to separate verification contracts.
Sandbox options	Reduce risk from tool execution and external actions.

3.2 DeepSeek V4 API Assumptions

The design assumes DeepSeek V4 is used as a generator and structured-output backend. DeepSeek documentation says its API format is compatible with OpenAI and Anthropic, with base URLs for both formats [R5]. It lists deepseek-v4-flash and deepseek-v4-pro as models, while legacy deepseek-chat and deepseek-reasoner are scheduled for deprecation on 2026-07-24 [R5].

The pricing/model page lists both V4 models as supporting thinking mode, 1M context length, JSON Output, and Tool Calls [R6]. Tool-call documentation states that the model returns function calls but does not execute functions; the user system provides the function behavior [R7]. That distinction is important: generated tool calls are proposals, not executed authority.

DeepSeek role	Recommended model	Reason
Complex candidate generation	deepseek-v4-pro	Use the stronger model when candidate quality matters.
Structured extraction	deepseek-v4-flash	Use cheaper model for deterministic JSON extraction and normalization.
Final answer synthesis	deepseek-v4-pro or flash	Select by task complexity and cost constraints.
Judge authority	External judge, not DeepSeek	The model can propose labels, but the fixed rule interpreter decides commits.

4. Core Design Thesis

The system is built around a strict authority separation:

LLM role: propose candidate moves
Adapter role: extract and normalize candidate moves
Judge role: classify moves under a verification contract
State role: commit only certified FOLLOW transitions
RAG/tool role: supply certificates for UNDECIDED moves

This is not a debate between several agents. It is a state-control architecture. The accepted state is not the transcript. The accepted state is a versioned data object mutated only by certified transitions.

$$J(H, m, V) \rightarrow \{ \text{FOLLOW, OMIT, NULL, UNDECIDED} \}$$

$$H_{\{t+1\}} = T_m(H_t) \text{ only when } J(H_t, m, V) = \text{FOLLOW}$$

5. Judge Semantics

Let H be the current verification state, m a normalized semantic move, V the verification contract, and T_m the candidate transition. The judge uses a fixed rule interpreter over certificates supplied by symbolic checks, retrieval evidence, tools, learned classifiers, or calibrated surrogates.

Symbol	Meaning
H	Current explicit verification state.
m	Normalized semantic move.
V	Verification contract.
$T_m(H)$	State produced by applying move m to H , if certified.
$\leq_V$	Information order under the contract.
$T_m(H) < H$	Strict refinement of current state.
$T_m(H) == H$	Semantic equivalence or invariance.

5.1 Fixed Rule

```

if Nullity( $H, m, V$ ) is certified:
    return NULL
if admissibility of  $m$  cannot be certified:
    return UNDECIDED
if  $T_m(H)$  cannot be certified:
    return UNDECIDED
if  $T_m(H)$  is certified inconsistent:
    return NULL
if  $T_m(H)$  is semantically equivalent to  $H$ :
    return OMIT
if  $T_m(H)$  strictly refines  $H$ :
    return FOLLOW
return UNDECIDED

```

5.2 Interpretation

FOLLOW is not stylistic relevance. It means certified state progress. OMIT is not falsity. It means admissible inessentiality or invariance. NULL is not mere uncertainty. It means certified inadmissibility or nullity. UNDECIDED is not a fourth logical truth value; it is an operational abstention caused by insufficient certification.

6. Target Architecture

```
User / Channel
  -> OpenClaw Gateway
    -> Judge-enabled Agent Workspace
      -> DeepSeek V4 Generator
        -> Candidate Adapter
          -> Judge Engine
            -> State Manager
              -> RAG / Tools / Verifiers
                -> Final Answer Synthesizer
```

The design requires an explicit pre-commit boundary. Every candidate move must pass through the judge before it can change state, trigger a non-read-only tool, or appear as accepted knowledge in the final answer.

Component	Responsibility	Must not do
OpenClaw Gateway	Channel ingress, routing, sessions, skill execution.	Decide logical validity.
DeepSeek Generator	Propose candidate moves and produce final text from accepted state.	Commit claims directly.
Adapter	Extract, normalize, and validate candidate moves.	Treat parsed JSON as semantically true.
Judge Engine	Classify moves under contract.	Invent missing evidence.
State Manager	Version state and commit FOLLOW transitions.	Accept transcript as state.
RAG/Tools	Supply evidence or executable certificates.	Execute destructive actions before approval.

7. OpenClaw Integration Design

7.1 Workspace Layout

```
~/ .openclaw/workspace/
AGENTS.md
SOUL.md
TOOLS.md
skills/
  judge-adapter/
    SKILL.md
  schemas/
    semantic_move.schema.json
    verdict.schema.json
    state.schema.json
  src/
    extractor.ts
    normalizer.ts
    judge_client.ts
    controller.ts
```

```

rag-verifier/
  SKILL.md
  src/
    retrieve.ts
    evidence_judge.ts
state-manager/
  SKILL.md
  src/
    store.ts
    transition.ts
    audit.ts

```

This layout follows the OpenClaw skill model: a skill is a folder containing a SKILL.md file with YAML front matter and markdown instructions [R2].

7.2 Agent Configuration Pattern

OpenClaw configuration supports per-agent skill allowlists and sandbox modes. Its configuration documentation shows skill defaults and per-agent overrides, and describes sandbox options such as off, non-main, and all [R4]. The judge agent should run with an explicit skill allowlist and sandboxing enabled for non-main or all sessions.

```

{
  agents: {
    defaults: {
      workspace: "~/.openclaw/workspace",
      skills: ["judge-adapter", "rag-verifier", "state-manager"],
      sandbox: {
        mode: "non-main",
        scope: "agent"
      }
    },
    list: [
      {
        id: "rag-judge",
        workspace: "~/.openclaw/workspace-rag",
        skills: ["judge-adapter", "rag-verifier", "state-manager"]
      },
      {
        id: "code-judge",
        workspace: "~/.openclaw/workspace-code",
        skills: ["judge-adapter", "state-manager"]
      }
    ]
  }
}

```

8. DeepSeek V4 API Integration

8.1 Generator Call

```

{
  "model": "deepseek-v4-pro",
  "messages": [
    {
      "role": "system",

```

```

    "content": "Propose candidate reasoning moves. Do not commit unsupported claims."
  },
  {
    "role": "user",
    "content": "Current state and user request here."
  }
],
"response_format": {"type": "json_object"}
}

```

8.2 Extraction Call

```

{
  "model": "deepseek-v4-flash",
  "messages": [
    {
      "role": "system",
      "content": "Extract atomic semantic moves from the draft. Return strict JSON only."
    },
    {
      "role": "user",
      "content": "Draft answer and current state here."
    }
  ],
  "response_format": {"type": "json_object"}
}

```

8.3 Tool Calls

DeepSeek tool-call documentation states that the model returns a function call, while the actual function implementation and execution are provided by the user system [R7]. Therefore, in this architecture a model-produced tool call is a candidate move. It must be validated and judged before execution.

```

Model emits: get_weather({"location": "Vienna"})
System interprets this as: SemanticMove(kind="tool_call", tool="get_weather")
Judge decides: FOLLOW, OMIT, NULL, or UNDECIDED
Controller executes only if policy permits the verdict

```

9. Candidate Adapter

The adapter is the highest-risk and highest-leverage component. Existing LLMs produce prose. The judge needs structured moves. The adapter converts raw text, structured JSON, tool calls, retrieval hits, or chain-of-thought-like drafts into normalized semantic moves.

```

raw LLM output
-> segmentation
-> claim extraction
-> operation classification
-> dependency linking
-> normalization
-> schema validation
-> judge input

```


Adapter stage	Output
Segmentation	Candidate spans or steps.
Claim extraction	Atomic claims or operations.
Operation classification	assert, derive, retrieve, calculate, cite, compare, tool_call, abstain.
Dependency linking	Evidence IDs, prior claim IDs, tool outputs, source spans.
Normalization	Typed semantic move object.
Schema validation	Accept/reject malformed structure before judge evaluation.

10. Verification State Model

The state must be external, explicit, versioned, and auditable. It must not be the raw conversation transcript. It should be compact enough to feed back to the model, but structured enough for the judge to compare transitions.

```
{
  "state_id": "s_001",
  "version": 12,
  "contract_id": "rag_fact_v1",
  "goal": "Answer the user question using accepted evidence only.",
  "accepted_claims": [],
  "omitted_claims": [],
  "null_claims": [],
  "undecided_claims": [],
  "evidence": [],
  "open_questions": [],
  "tool_results": [],
  "audit_log": []
}
```

10.1 Semantic Move Schema

```
{
  "move_id": "mv_001",
  "kind": "assert_claim",
  "surface_text": "The cited passage supports claim X.",
  "claim_id": "cl_001",
  "dependencies": ["ev_001"],
  "target_variables": ["q_2"],
  "assertion_mode": "established",
  "contract_id": "rag_fact_v1"
}
```

11. Judge Engine

The judge engine should be implemented as a local service or internal OpenClaw tool. It should expose a stable API independent of the LLM provider. The first MVP can use a rule-based or hybrid judge; a learned discriminative judge can be added later.

Judge submodule	Purpose
Rule interpreter	Fixed mapping from certificates to verdicts.
Contract loader	Loads contract-specific schemas, thresholds, and rules.
Certificate evaluator	Combines symbolic rules, RAG evidence, tools, and learned scores.
Risk controller	Applies confidence, calibration, and coverage policy.
Transition builder	Creates state delta for FOLLOW.
Audit writer	Records verdict, reason code, confidence, and certificate.

11.1 Verdict Object

```
{
  "move_id": "mv_001",
  "verdict": "UNDECIDED",
  "reason_code": "INSUFFICIENT_EVIDENCE",
  "confidence": 0.62,
  "certificate": null,
  "transition": null,
  "required_actions": ["retrieve"]
}
```

12. RAG and Tool Integration

RAG should be driven by UNDECIDED moves rather than raw similarity. A passage is not accepted because it is topically similar. It is accepted only if it creates a certified state refinement or supplies a certificate required by a candidate.

```
candidate c -> judge
if verdict == UNDECIDED:
    retrieve evidence R
    judge R as evidence move
    if J(H, R) == FOLLOW:
        H1 = T_R(H)
        rejudge c under H1
```

For tool use, the same principle applies. A tool call emitted by the model is a candidate move. Read-only tools may be judged under a lower-risk policy; write or destructive tools require stronger approval and should not execute on UNDECIDED.

13. Internal API Specification

13.1 POST /adapter/extract

```
Request:
{
  "state": {...},
  "draft": "raw model output"
}

Response:
{
  "moves": [ {...}, {...} ],
  "warnings": []
}
```

13.2 POST /judge/evaluate

```
Request:
{
  "state": {...},
  "move": {...},
  "contract": {...}
}

Response:
{
  "verdict": "FOLLOW",
  "reason_code": "SUPPORTED_REFINEMENT",
  "confidence": 0.94,
  "transition": {...},
  "certificate": {...}
}
```

13.3 POST /state/commit

```
Request:
{
  "state_id": "s_001",
  "move_id": "mv_001",
  "verdict_id": "v_001",
  "transition": {...}
}

Response:
{
  "new_state_id": "s_002",
  "status": "committed"
}
```

14. Storage Model

The MVP can use SQLite with JSON columns. Later versions can move to PostgreSQL plus vector indexes and object storage. The storage layer must support versioning, rollback, and audit.

```

CREATE TABLE states (
  state_id TEXT PRIMARY KEY,
  session_id TEXT NOT NULL,
  contract_id TEXT NOT NULL,
  version INTEGER NOT NULL,
  state_json TEXT NOT NULL,
  created_at TEXT NOT NULL
);

CREATE TABLE moves (
  move_id TEXT PRIMARY KEY,
  state_id TEXT NOT NULL,
  kind TEXT NOT NULL,
  move_json TEXT NOT NULL,
  source TEXT NOT NULL,
  created_at TEXT NOT NULL
);

CREATE TABLE verdicts (
  verdict_id TEXT PRIMARY KEY,
  move_id TEXT NOT NULL,
  state_id TEXT NOT NULL,
  verdict TEXT NOT NULL,
  reason_code TEXT NOT NULL,
  confidence REAL,
  certificate_json TEXT,
  created_at TEXT NOT NULL
);

```

15. Security Model

The security model treats every model output, webhook payload, retrieval result, and external tool response as untrusted input. This is consistent with OpenClaw configuration guidance for webhook payloads, which states that webhook content should be treated as untrusted input and recommends dedicated hook tokens and strict tool policy [R4].

- No generated tool action executes before controller approval.
- No memory write occurs before a FOLLOW verdict.
- No destructive tool call may execute on UNDECIDED.
- NULL actions are blocked and logged.
- Every state mutation must have a verdict ID and certificate or reason code.
- OpenClaw sandboxing should be enabled at least for non-main sessions; all-session sandboxing is preferred for risky tools.

Security invariant

The accepted state is a security boundary. The transcript is not trusted state. Retrieval output is not trusted evidence until judged.

16. Cost Model

The system will be more expensive than a direct single-call LLM baseline. The goal of the MVP is not immediate cost superiority; it is measurement of safety and state-control value. Cost can be optimized after the architecture demonstrates lower false-follow rate.

$$C_{\text{turn}} = C_G + k C_A + k C_J + m C_R + r C_{\text{rejudge}} + C_F$$

Term	Meaning
C_G	DeepSeek generation cost.
C_A	Adapter extraction and normalization cost.
C_J	Judge evaluation cost.
C_R	Retrieval or tool-verification cost.
C_rejudge	Cost of re-evaluating moves after state updates.
C_F	Final answer synthesis cost.
k	Number of candidate moves.
m	Number of retrieval/tool calls.
r	Number of rejudgments.

DeepSeek pricing is token-based, with listed input/output token rates and model details on the official pricing page [R6]. Actual cost must be measured in the deployment environment because the judge architecture multiplies calls.

17. Evaluation Plan

The primary evaluation is not answer elegance. The primary evaluation is whether the system prevents invalid or unsupported moves from entering accepted state.

Metric	Definition	Target direction
FalseFollowRate	$P(J=\text{FOLLOW and move is not a valid refinement})$.	Decrease aggressively.
FalseNullRate	$P(J=\text{NULL and move is not actually null/inadmissible})$.	Decrease without raising false follow.
Coverage	Fraction of moves receiving non-UNDECIDED verdicts.	Increase only under controlled risk.
UnsupportedClaimRate	Final-answer claims without accepted support.	Decrease.
RetrievalWasteRate	Retrieved items judged OMIT or NULL.	Decrease.
CostPerCorrectAnswer	Total system cost divided by correct audited answers.	Monitor.

Metric	Definition	Target direction
LatencyPerTurn	End-to-end turn latency.	Monitor.

17.1 Baselines

- DeepSeek V4 direct answer.
- DeepSeek V4 with standard RAG.
- DeepSeek V4 with self-check prompt.
- OpenClaw agent without judge middleware.
- OpenClaw agent with judge adapter and explicit state.

18. Implementation Plan

Phase	Duration	Deliverables
0. Architecture spike	1-2 weeks	OpenClaw running, DeepSeek client, state store skeleton, manual judge hook.
1. Candidate extraction	2-3 weeks	JSON schemas, extractor prompt, validation, move logs.
2. Rule-based RAG judge	3-4 weeks	Controlled corpus, evidence linking, verdict rules.
3. Hybrid judge	4-6 weeks	NLI/contradiction module, calibration, batch judge API.
4. Evaluation harness	2-4 weeks	Benchmarks, baselines, cost/latency dashboards.
5. Learned judge	6-10 weeks	Training data, small discriminative model, risk-coverage report.

19. Recommended MVP

The recommended MVP is controlled factual RAG over a fixed document corpus. This is natural-language enough to be meaningful, while still allowing objective evaluation of supported, contradicted, redundant, and undecidable claims.

- **Input:** User question over a controlled corpus.
- **Generator:** DeepSeek V4 proposes candidate claims and answer outline.
- **Adapter:** Extracts atomic claims and evidence requirements.
- **Judge:** Classifies claim/evidence moves.
- **RAG:** Runs only for UNDECIDED moves or explicit evidence gaps.
- **Finalizer:** Produces answer from accepted claims only.
- **Audit:** Every final claim maps to accepted state or is removed.

20. Risk Register

Risk	Impact	Mitigation
Adapter hallucination	Invented moves corrupt the judgment pipeline.	Use span grounding, schema validation, and draft-offset references.
Over-nullification	Open-world unknowns get incorrectly blocked.	Separate proposition from assertion act; use UNDECIDED for absence of evidence.
False follow	Invalid move enters accepted state.	High thresholds, rollback, conservative authority policy, audit.
Cost explosion	Too many calls and judgments.	Candidate caps, batching, caching, flash model for extraction.
Tool execution risk	Generated actions affect external systems.	Pre-commit judge gate, sandbox, allowlists, no destructive tools in MVP.
Judge learns style	Judge becomes preference model.	Train/evaluate on counterfactual and contract-specific labels.
State bloat	State becomes too large for efficient prompting.	Summarize accepted state; keep audit log external.
Contract ambiguity	Verdicts inconsistent across domains.	One explicit contract per domain; no universal fallback in MVP.

Appendix A. Minimal OpenClaw Skill Skeleton

```
---
name: judge-adapter
description: Extracts candidate semantic moves and routes them through the verification judge
before state update or tool execution.
---
```

```
# Judge Adapter Skill
```

```
Use this skill whenever the assistant proposes claims, reasoning steps, retrieved evidence,
or tool actions.
```

```
Pipeline:
1. Extract semantic moves.
2. Validate move schema.
3. Evaluate each move through the judge.
4. Commit FOLLOW only.
5. Route UNDECIDED to retrieval or verification.
6. Reject NULL.
7. Drop or compress OMIT.
```

```
Never execute destructive tool calls without an approved verdict under the relevant contract.
```

Appendix B. Minimal Verification Contract

```
{
  "contract_id": "rag_fact_v1",
  "state_schema": "FactualRagState",
  "candidate_schema": "SemanticMove",
  "allowed_operations": [
    "assert_claim",
    "link_evidence",
    "derive",
    "retrieve",
    "summarize",
    "reject"
  ],
  "authority_policy": {
    "min_follow_confidence": 0.90,
    "min_null_confidence": 0.92,
    "require_citation_for_assertion": true,
    "allow_open_world_undecided": true
  }
}
```

Appendix C. References

- [R1] OpenClaw Docs, "What is OpenClaw?" and key capabilities. <https://docs.openclaw.ai/>
- [R2] OpenClaw Skills documentation.
<https://github.com/openclaw/openclaw/blob/main/docs/tools/skills.md>
- [R3] OpenClaw Multi-agent routing documentation. <https://docs.openclaw.ai/concepts/multi-agent>
- [R4] OpenClaw Gateway configuration documentation.
<https://docs.openclaw.ai/gateway/configuration>
- [R5] DeepSeek API Docs, "Your First API Call". <https://api-docs.deepseek.com/>
- [R6] DeepSeek API Docs, "Models & Pricing". https://api-docs.deepseek.com/quick_start/pricing/
- [R7] DeepSeek API Docs, "Tool Calls". https://api-docs.deepseek.com/guides/tool_calls/